

// Jivops

Guía completa — 30 lecciones

Arquitectura · Transformadores básicos · Patches avanzados ·
Generadores · Estructura y componentes · Integración avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Kustomize y el enfoque template-free
- Lección 02 — kustomization.yaml — estructura básica
- Lección 03 — Instalación y uso (kubectl -k, standalone binary)
- Lección 04 — Bases y overlays — el patrón fundamental
- Lección 05 — Kustomize vs Helm — cuándo usar cada uno

Módulo 2: Transformadores básicos

- Lección 06 — commonLabels y commonAnnotations
- Lección 07 — namePrefix y nameSuffix
- Lección 08 — namespace transformer
- Lección 09 — images — sobrescritura de tags de imagen
- Lección 10 — replicas — ajuste de réplicas por overlay

Módulo 3: Patches avanzados

- Lección 11 — strategicMergePatch — fundamentos
- Lección 12 — JSONPatch (RFC 6902) — patches precisos
- Lección 13 — patchesStrategicMerge vs patches (sintaxis moderna)
- Lección 14 — Patches condicionales por target (labelSelector, annotationSelector)
- Lección 15 — Inline patches vs patches en archivos separados

Módulo 4: Generadores

- Lección 16 — ConfigMapGenerator — generación de ConfigMaps
- Lección 17 — SecretGenerator — generación de Secrets
- Lección 18 — Opciones de generadores — disableNameSuffixHash
- Lección 19 — Generadores desde archivos externos (envs, files)
- Lección 20 — Generadores personalizados (KRM functions)

Módulo 5: Estructura de proyecto y componentes

- Lección 21 — Estructura de directorios recomendada (base/overlays)
- Lección 22 — Components — piezas reutilizables entre overlays
- Lección 23 — Remote bases — referenciar bases desde un repositorio Git
- Lección 24 — Composición de múltiples bases
- Lección 25 — Validación de manifiestos generados (kustomize build | kubectl apply --dry-run)

Módulo 6: Integración avanzada

Lección 26 — Kustomize con ArgoCD/Flux (GitOps)

Lección 27 — Gestión de secretos con Kustomize (SOPS, Sealed Secrets)

Lección 28 — Testing de overlays antes de aplicar

Lección 29 — Migración de Helm a Kustomize (o combinación de ambos)

Lección 30 — Proyecto final: estructura de despliegue multi-entorno completa con Kustomize lista para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Kustomize y el enfoque template-free

Los manifiestos base son YAML legible directamente, sin sintaxis de templating que interpretar mentalmente antes de saber el resultado final.

Lección 02 — kustomization.yaml — estructura básica

Da control explícito y predecible sobre qué se incluye y qué transformaciones se aplican, en vez de que Kustomize adivine la intención.

```
resources: - deployment.yaml
```

Lección 03 — Instalación y uso (kubectl -k, standalone binary)

Separar 'kustomize build' de 'kubectl apply' permite inspeccionar el YAML generado antes de confirmar que realmente se aplique.

```
kustomize build ./overlays/production
```

Lección 04 — Bases y overlays — el patrón fundamental

La base captura la configuración compartida una vez; cada overlay solo declara las diferencias específicas de su entorno.

```
bases: - ../../base
```

Lección 05 — Kustomize vs Helm — cuándo usar cada uno

La lógica condicional de Helm es más apropiada para un chart público con necesidades muy variadas que el enfoque más directo de Kustomize.

Módulo 2: Transformadores básicos

Lección 06 — commonLabels y commonAnnotations

Kustomize actualiza automáticamente los selectors relacionados, evitando que un Deployment deje de encontrar sus propios Pods tras la transformación.

```
commonLabels: environment: production
```

Lección 07 — namePrefix y nameSuffix

Un prefijo distintivo evita colisión de nombres si dos entornos comparten el mismo namespace por alguna razón particular.

```
namePrefix: staging-
```

Lección 08 — namespace transformer

La base permanece genérica sin namespace hardcodeado; cada overlay decide su propio destino, permitiendo reutilizar la misma base.

```
namespace: produccion
```

Lección 09 — images — sobrescritura de tags de imagen

El pipeline sobrescribe solo el tag en el overlay específico, sin modificar (y potencialmente romper) el manifiesto compartido de la base.

```
newTag: "1.2.3"
```

Lección 10 — replicas — ajuste de réplicas por overlay

Desarrollo típicamente necesita mucha menos capacidad real que producción, así que menos réplicas ahorra recursos sin sacrificar nada relevante.

```
count: 5
```

Módulo 3: Patches avanzados

Lección 11 — strategicMergePatch — fundamentos

Kustomize fusiona el patch con la base conservando lo no mencionado, así que solo hace falta declarar el campo específico a cambiar.

Lección 12 — JSONPatch (RFC 6902) — patches precisos

Eliminar un elemento específico por posición en un array es una operación que JSONPatch expresa de forma directa, distinta al enfoque de merge.

```
"op": "replace"
```

Lección 13 — patchesStrategicMerge vs patches (sintaxis moderna)

El campo unificado 'patches' reduce la sintaxis distinta a recordar, consolidando ambos tipos de patch bajo un solo concepto consistente.

```
patches: - path: mi-patch.yaml
```

Lección 14 — Patches condicionales por target (labelSelector, annotationSelector)

Un solo patch con labelSelector cubre automáticamente todos los recursos que coincidan, actuales y futuros, sin repetir por cada uno.

```
labelSelector: "tier=backend"
```

Lección 15 — Inline patches vs patches en archivos separados

Un cambio pequeño no necesita archivo aparte; un patch grande y complejo se beneficia de estar separado para mantener el kustomization.yaml legible.

```
path: patches/resource-limits.yaml
```

Módulo 4: Generadores

Lección 16 — ConfigMapGenerator — generación de ConfigMaps

El nombre generado incluye un hash del contenido; al cambiar el nombre referenciado, Kubernetes dispara automáticamente un rolling update de los Pods.

```
literals: - clave=valor
```

Lección 17 — SecretGenerator — generación de Secrets

Un valor sensible como literal en kustomization.yaml quedaría expuesto en el historial de Git; un archivo externo puede excluirse aparte.

```
files: - password.txt
```

Lección 18 — Opciones de generadores — disableNameSuffixHash

Sin el hash, los Pods no detectan automáticamente que la configuración cambió, pudiendo quedar corriendo con una versión desactualizada.

```
generatorOptions: disableNameSuffixHash: true
```

Lección 19 — Generadores desde archivos externos (envs, files)

'files' toma el archivo completo intacto como valor único, útil para un nginx.conf entero que no sigue el formato clave=valor de 'envs'.

```
envs: - config.env
```

Lección 20 — Generadores personalizados (KRM functions)

Una organización con una transformación muy específica puede escribir su propia KRM function, similar a un plugin que extiende Kustomize.

Módulo 5: Estructura de proyecto y componentes

Lección 21 — Estructura de directorios recomendada (base/overlays)

Seguir la convención estándar facilita que un nuevo miembro del equipo entienda la organización sin explicación adicional extensa.

```
overlays/production/
```

Lección 22 — Components — piezas reutilizables entre overlays

Un Component reutilizable de 'métricas' evita copiar la misma configuración en cada overlay, permitiendo actualizarla en un solo lugar.

```
kind: Component
```

Lección 23 — Remote bases — referenciar bases desde un repositorio Git

Fijar una versión específica al usar una base remota evita que un cambio inesperado en la rama principal afecte tu propio despliegue.

```
github.com/miorg/mirepo/base?ref=v1.0.0
```

Lección 24 — Composición de múltiples bases

Bases pequeñas y especializadas son más fáciles de entender y reutilizar de forma independiente que una única base monolítica gigante.

```
- ../base-monitoring
```

Lección 25 — Validación de manifiestos generados (kustomize build | kubectl apply --dry-run)

'--dry-run=server' consulta al servidor real de la API, dando una validación más completa que solo verificar sintaxis localmente.

```
--dry-run=server
```


Módulo 6: Integración avanzada

Lección 26 — Kustomize con ArgoCD/Flux (GitOps)

ArgoCD ejecuta 'kustomize build' automáticamente en su reconciliación, eliminando el riesgo de que alguien olvide regenerar el YAML manualmente.

Lección 27 — Gestión de secretos con Kustomize (SOPS, Sealed Secrets)

Un archivo cifrado con SOPS versionado en Git es ilegible sin la clave correspondiente, a diferencia de un secreto en texto plano.

```
sops --encrypt --in-place secreto.yaml
```

Lección 28 — Testing de overlays antes de aplicar

Comparar contra un snapshot esperado detecta si un cambio en la base afectó accidentalmente a un overlay que no debía verse afectado.

```
kustomize build ./overlays/prod > expected.yaml
```

Lección 29 — Migración de Helm a Kustomize (o combinación de ambos)

Helm maneja la lógica compleja del chart; Kustomize aplica de forma simple los ajustes finales por entorno sobre ese resultado renderizado.

```
helm template mi-chart | kustomize build -
```

Lección 30 — Proyecto final: estructura de despliegue multi-entorno completa con Kustomize lista para producción

Se integra todo lo anterior: base clara, overlays por entorno, generadores para config/secrets, Components reutilizables, integración GitOps con ArgoCD/Flux y validación automatizada — el nivel esperado de una estructura de despliegue con Kustomize lista para producción real.